Report on Algorithms in linux/kernel/shed

Firstly, let's go through all the files. I see that the **scheduler** is split into some **scheduling classes**, and most of the classes have their own algorithm.

Secondly, every CPU has its own **runqueue** (**struct rq**) with a smaller queue for each class. When a CPU needs a new task, **__schedule()** (in **core.c**) asks the classes in a fixed order and runs the first task it gets. This fixed order is based on the priority.

stop → deadline → real-time → fair → ext → idle
stop_task.c → deadline.c → rt.c → fair.c → ext/ext.c → idle.c
(-) → SCHED_DEADLINE → SCHED_FIFO, SCHED_RR → SCHED_NORMAL, SCHED_BATCH, SCHED_IDLE → SCHED_EXT → (-)

Algorithms used there:

1) **Preemptive Priority scheduling** (between the classes)
   The CPU always runs the task with the highest priority. Here, Linux uses it to choose between the classes. The priority was shown above. It's preemptive: if a higher-priority task wakes up, it takes the CPU away.
   Files: rt.c, core.c
   Used for: choosing which class runs, and ordering real-time priorities.

2) **FIFO, FCFS**
   Inside one priority level tasks run in the order they arrived. A task keeps the CPU until it blocks or gives it up.
   Usage: rt.c, SCHED_FIFO

3) **Round Robin**
   Like FIFO, but each task gets a quantum (default 100 ms). task_tick_rt() counts it down, and at 0 the task goes to the end of its queue.
   Usage: rt.c, SCHED_RR

4) **EDF** (Earliest Deadline First)
   Each task gives a runtime, a deadline and a period. And the closest deadline runs first. **sched_dl_overflow()** accepts a new task only if the total load still fits.
   Usage: deadline.c, SCHED_DEADLINE

5) **EEVDF** (Earliest Eligible Virtual Deadline First)
   Previously it was **CFS** (Completely Fair Scheduler). But now it was replaced with **EEVDF**, where firstly eligible tasks with the earliest virtual deadline run. The eligible ones are those that got less than their fair share.
   Usage: fair.c, SCHED_NORMAL, SCHED_BATCH, SCHED_IDLE